

SE 811: Software Maintenance

Toukir Ahammed

Adaptive Change

- Adaptive change is a change introduced to accommodate any modifications in the environment of a software product.
- The term environment here refers to the totality of all conditions that influence the software product, including business rules, government policies, and software and hardware operating systems.
- For example, when a library or platform developer may evolve its APIs, the corresponding adaptation may be required for client applications to handle such environment change.
- As another example, when porting a mobile application from Android to iOS, mobile developers need to apply adaptive changes to translate the code from Java to Swift, so that the software is still compilable and executable on the new platform.

Adaptive Change

We will focus on three scenarios of adaptive changes:

1. Cross-system software porting,
2. Cross-language software migration, and
3. Software library upgrade (i.e., API evolution).

Cross-System Porting

Consider an example of cross-system porting.

- When a software system is installed on a computer, the installation can depend on the configurations of the hardware, the software, and the device drivers for particular devices.
- To make the software to run on a different processor or an operating system, and to make it compatible with different drivers, we may need adaptive changes to adjust the software to the new environment.

Cross-System Porting

- Software forking—creating a variant product by copying and modifying an existing product—is often considered an ad hoc, low-cost alternative to principled product line development.
- To maintain such forked products, developers often need to port an existing feature or bug-fix from one product variant to another.

Empirical Studies on Cross-System Porting

- OpenBSD, NetBSD, and FreeBSD have evolved from the same origin BSD but have been maintained independently from one another.
- The extent and nature of cross-system porting in the BSD family:
 - (1) the information flow among the forked BSD family is decreasing;
 - (2) 40% of lines of code were shared among the BSD family;
 - (3) in some modules such as device driver modules, there is a significant amount of adopted code; and
 - (4) contributors who port changes from other projects are highly active contributors

Empirical Studies on Cross-System Porting

- Maintaining forked projects involves significant effort of porting patches from other projects—10–15% of patch content was ported from another project's patches.
- Cross-system porting is periodic, and its rate does not necessarily decrease over time.
- Ported changes are less defect-prone than non-porting changes.
- While most ported changes migrate to peer projects in a relatively short amount of time, some changes take a very long time to propagate to other projects.
- Ported changes are localized within less than 20% of the modified files per release on average in all three BSD projects, indicating that porting is concentrated on a few subsystems.

Cross-Language Migration

- Consider an example of cross-language migration where you have software in Java that must be translated to C. Developers need to rewrite software and must also update language-specific libraries.
- When maintaining a legacy system that was written in an old programming language (e.g., Fortran) decades ago, programmers may migrate the system to a mainstream general-purpose language, such as Java, to facilitate the maintenance of existing codebase and to leverage new programming language features.

Cross-Language Program Translation

- To translate code implementation from one language to another, researchers have built tools by hard coding the translation rules and implementing any missing functionality between languages.
- For example,
 - automating the COBOL-to-Java code migration by defining and implementing rules to generate Java classes, methods, and packages from COBOL programs.
 - automating the Java-to-C# migration using a phrase-based statistical machine translation approach
 - encoding both Java and C# source files into sequences of syntactic symbols, and then relies on it to align code and to train sequence-to-sequence translation.

Mining Cross-Language API Rules

- When migrating software to a different target language, API conversion poses a challenge for developers, because the diverse usage of API libraries induces an endless process of specifying API translation rules or identifying API mappings across different languages.
- Researchers have built tools that can
 - automatically mine API usage mappings between Java and C#, align code based on similar names and then construct the API transformation graphs for each pair of aligned statements.
 - automatically mine API usage sequence mappings by conducting program dependency analysis and representing API usage as a graph-based model.

Library Upgrade and API Evolution

- Instead of building software from scratch, developers often use existing frameworks or third-party libraries to reuse well-implemented and tested functionality.
- Ideally, the APIs of libraries must remain stable such that library upgrades do not incur corresponding changes in client applications.
- In reality, however, APIs change their input and output signatures, change semantics, or are even deprecated, forcing client application developers to make corresponding adaptive changes in their applications.

Library Upgrade and API Evolution

Consider the example of API evolution.

- When the APIs of a library and a platform evolve, corresponding adaptations are often required for client applications to handle such API update.
- In extreme cases, e.g., when porting a Java desktop application to the iOS platform, developers need to rewrite everything from scratch, because both the programming language (i.e., Swift) and software libraries are different.

Empirical Studies of API Evolution

- API changes may lead to subsequent changes on dependent drivers, and such collateral evolution could introduce bugs into previously mature code.
- 14% of deprecated methods produce nontrivial API change effects in at least one client-side project; however, these effects vary greatly in magnitude.
- On average, a single API deprecation resulted in 5 broken projects, while the largest caused 79 projects and 132 packages to break

Tool Support for API Evolution and Client Adaptation

- Several existing approaches semi-automate or automate client adaptations to cope with evolving libraries.
- CatchUp (*record and replay*):
 - assumes that the adaptation changes in client code are exact or similar to the changes in the library side
 - records and stores refactoring in an XML file that can be replayed to update client code
 - limited to three refactoring: renaming operations (e.g., types, methods, fields), moving operations (e.g., classes to different packages, static members), or change operations (e.g., types, signatures).

Tool Support for API Evolution and Client Adaptation

- Diff-CatchUp: automatically recognizes API changes of the reused framework and suggests plausible replacements to the obsolete APIs based on the working examples of the framework codebase
- SemDiff: mines API usage changes from other client applications or the library itself. It defines an adaptation pattern as a frequent replacement of a method invocation. That is, if a method call to *A.m* is changed to *B.n* in several adaptations, *B.n* is likely to be a correct replacement for the calls to *A.m*.